

Assignment 6

GUI-Based Multi-Client Chat Application Using TCP

Networking Internship — Phase 3

Tezpur University

Submitted by: Pipal Ratneswar Deka

Roll No.: CSb24040

Date: 11 July 2026

1. Objective

The objective of this assignment is to convert the terminal-based TCP chat application developed in Assignment 5 into a graphical desktop application, while reusing the existing server implementation without modification. The exercise introduces GUI programming with Tkinter, event-driven design, multithreading for non-blocking network I/O, and user-friendly network application development. The server (server.py) continues to run unmodified from Assignment 5; only the client is rebuilt as a GUI application (client_gui.py) that reuses the exact same TCP socket protocol — broadcast messaging, private messaging, online user listing, connection/disconnection notifications, and server-side chat history/logging.

Network Setup: The system was tested in Mininet using a single-switch, five-host topology:

```
sudo mn --topo single,5
```

Host	Role
h1	Chat Server (server.py, port 5000)
h2	Client A
h3	Client B
h4	Client C
h5	Client D

Connectivity between all hosts was verified using the nodes, net, and pingall commands inside the Mininet CLI before running the server and client applications.

2. GUI Design

The client GUI is built with Tkinter/ttk and consists of two screens managed within a single Tk root window: a Login window and a Chat window. The login screen collects and validates a username before a socket connection is attempted; on success, the login frame is destroyed and replaced in-place by the chat frame.

2.1 Login Window

The login window contains a title label, a username entry box (letters only), an inline status/error label, a Connect button, and a hint showing the server address (10.0.0.1:5000). Empty usernames and usernames containing anything other than letters are rejected client-side with an inline error message before any socket connection is attempted.

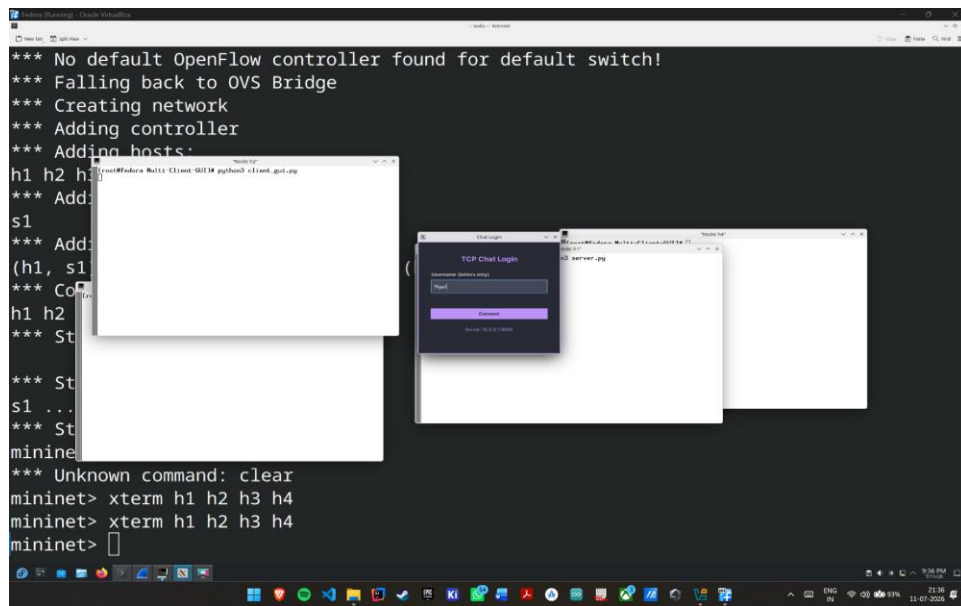


Figure 2.1 — Login window with username entry and Connect button

2.2 Successful Connection

On pressing Connect, the button is disabled and the status label shows "Connecting..." while the socket handshake completes. Once the server accepts the connection, the label updates to "Connected!" in green before the chat window is shown.

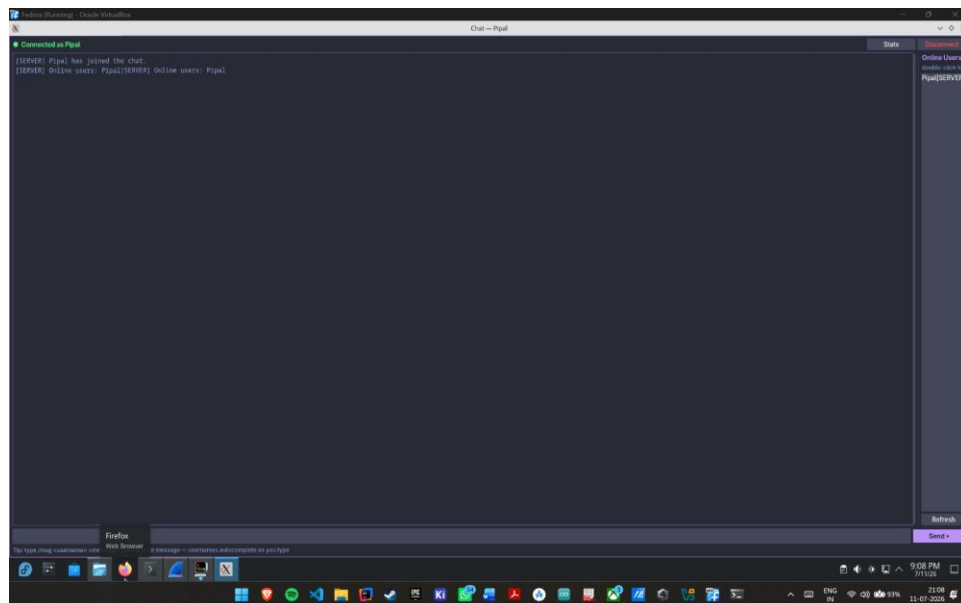


Figure 2.2 — Status label confirming a successful connection

2.3 Main Chat Window

The chat window (780x520) is split into a status bar (top), a scrollable chat display + online-users panel (body), and a message input row (bottom). The window title updates to "Chat — <username>" and the status bar shows a green "● Connected as <username>" indicator alongside a Disconnect button.

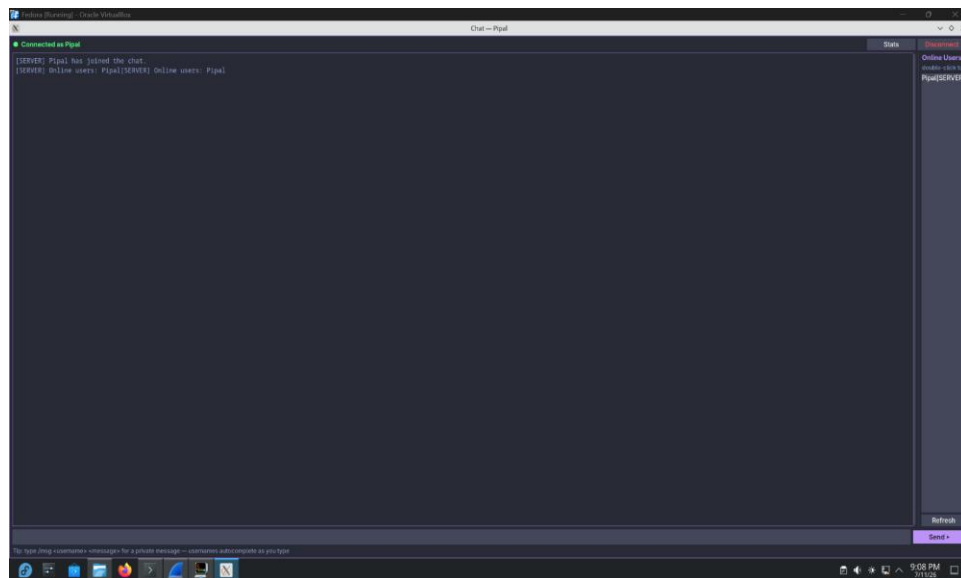


Figure 2.3 — Main chat window: message history, online users list, input box, Send/Disconnect buttons

2.4 Broadcast Messaging

Plain text typed into the input box (not starting with a command) is sent as a normal broadcast message. The server relays it to every other connected client, and the sender's own copy is echoed locally by the client the moment Send is pressed.

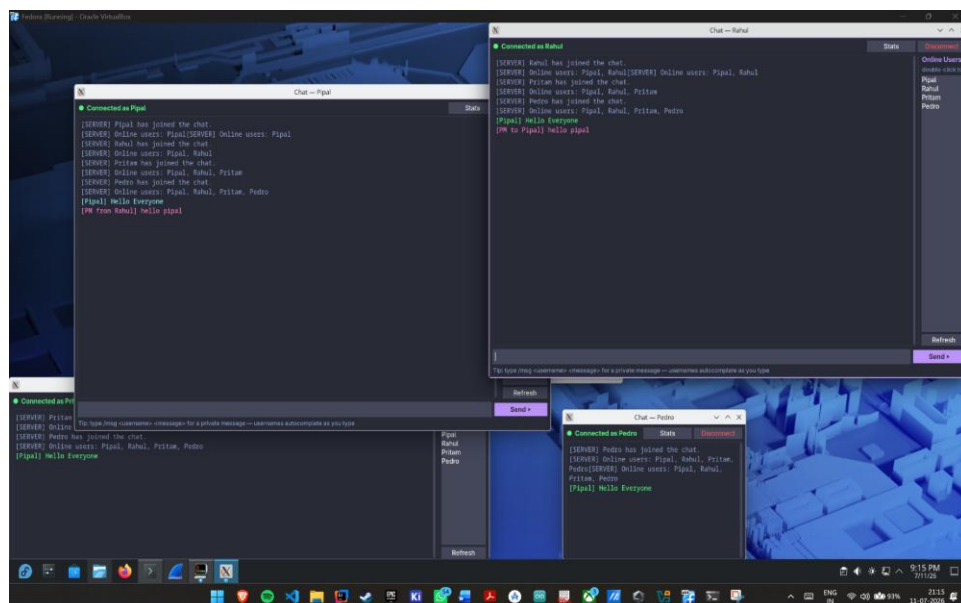
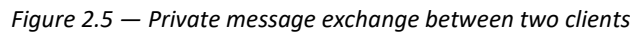


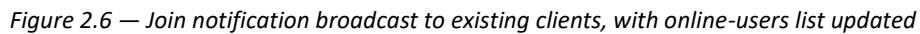
Figure 2.4 — Broadcast message visible to all connected clients

2.5 Private Messaging

Private messages are sent with the `/msg <username> <message>` syntax, or by double-clicking a name in the online-users list, which pre-fills the input box. As the user types after `/msg`, a small autocomplete popup lists matching online usernames. Private messages are rendered in a distinct color (pink) in the chat display.



Whenever a user connects or disconnects, the server broadcasts a system notification to all remaining clients, and every client's online-users panel is refreshed automatically.



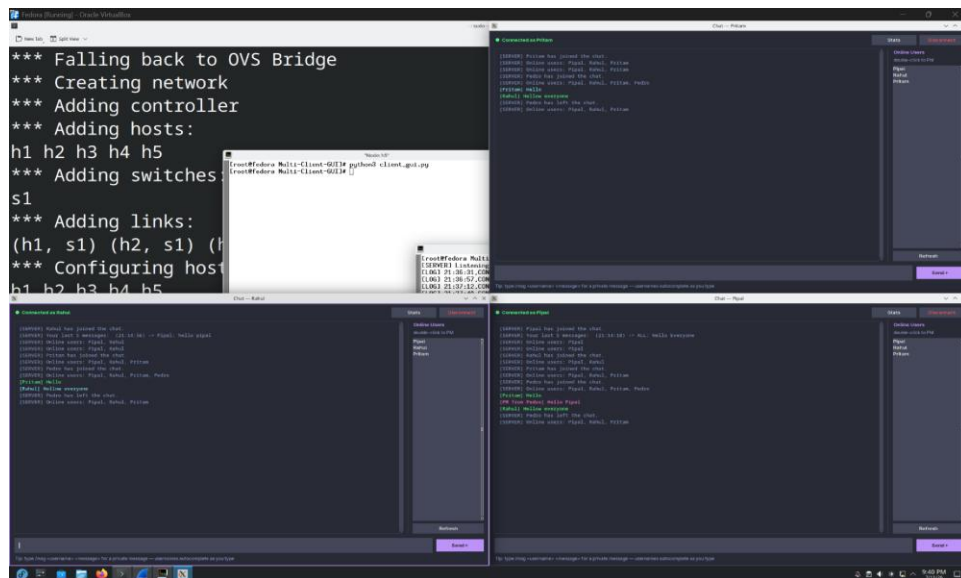


Figure 2.7 — Leave notification broadcast after a client disconnects

3. System Architecture

The application follows a classic multi-threaded client–server TCP architecture:

- Server (server.py): A single threaded TCP server listening on 0.0.0.0:5000. For every accepted connection, a new daemon thread (handle_client) is spawned to service that client independently, so the server can handle all four Mininet clients (h2–h5) concurrently.
- Shared state: An in-memory clients dictionary (socket → {username, ip, port, login_time, status}) guarded by a threading.Lock tracks who is online; a separate stats dictionary (also lock-protected) tracks message counters for the /stats command.
- Persistence: Every chat message is appended to chat_history.csv (timestamp, sender, receiver, type, message), and every connect/disconnect event is appended to server_logs.txt — both reused unchanged from Assignment 5.
- Client (client_gui.py): Splits cleanly into a networking layer (the ChatConnection class) and a presentation layer (Tkinter frames). ChatConnection owns the raw socket and a background daemon thread that blocks on recv() in a loop, pushing every decoded line onto a thread-safe queue.Queue. It never touches any Tkinter widget directly.
- GUI polling loop: The Tkinter main loop calls root.after(100, self._poll_incoming) to drain the queue every 100 ms on the GUI thread and update widgets (chat log, online-users list) accordingly. This keeps all Tkinter calls on the main thread — which Tkinter requires — while all blocking socket I/O happens off it.

Data flow for a typical message:

```
User types message -> send_message() -> conn.send(text) -> socket.send()
Server recv() -> parses command/broadcast -> broadcast()/safe_send() to target(s)
Target client's recv thread -> queue.put(data) -> _poll_incoming() -> _handle_line() -> chat_display
```

4. Components Reused from Assignment 5

Component	Reused as-is?	Notes
server.py (entire file)	Yes, unmodified	Same TCP server, thread-per-client model, and command set (/list, /stats, /msg, broadcast)
Broadcast messaging logic	Yes	broadcast() / broadcast_all() functions on the server are untouched
Private messaging (/msg)	Yes	Server-side find_socket_by_username() + formatted [PM ...] delivery reused directly
Online user management	Yes	clients dict + clients_lock reused; GUI only adds a visual listbox on top
Chat history logging (CSV)	Yes	write_history()/get_last_messages() unchanged — GUI client still receives the "last 5 messages" replay on reconnect
Server event logging	Yes	write_event_log() to server_logs.txt unchanged
Socket send/recv protocol	Yes	Same plain-text line protocol over the same TCP socket API
Terminal I/O (input()/print())	No — replaced	Replaced by Tkinter widgets and a background receiver thread

5. GUI Design Decisions

- Single-file client with clear separation: ChatConnection (networking) has zero Tkinter imports and communicates only through a queue.Queue, keeping networking logic testable and independent of the UI, even though both live in client_gui.py for simplicity.
- Nord color palette: A consistent dark theme (background #2E3440, accent #88C0D0, etc.) is applied via a single apply_nord_theme() function using ttk.Style, plus manual color configuration for classic Tk widgets (Listbox, ScrolledText, Entry) that ttk.Style cannot reach.
- Color-coded message tags: The chat display uses Text widget tags to color system messages, private messages, errors, the user's own sent messages, and broadcast messages differently, so the type of each line is visible at a glance.
- Autocomplete for /msg: A lightweight borderless Toplevel (SuggestionPopup) lists online usernames matching what is typed after "/msg ", navigable with Up/Down/Enter/Escape, reducing typos when addressing private messages.
- Double-click-to-PM: Double-clicking a name in the online-users list pre-fills "/msg <name> " in the input box as a faster alternative to typing the command.
- Non-blocking network I/O: All socket recv() calls happen on a dedicated background thread; the GUI thread only ever reads from a thread-safe queue via a 100 ms polling loop (root.after), so the window never freezes even while waiting for server data.
- Graceful disconnect handling: If the server closes the connection or the socket errors out, a special DISCONNECT_SIGNAL is pushed onto the queue, which the GUI turns into a status-label update and a warning dialog instead of crashing.

6. Testing Results

The application was tested with four simultaneous GUI clients (Pipal, Rahul, Pritam, Pedro) connected to the server running on h1, inside Mininet, using separate xterm windows for each host.

Test Case	Result	Evidence
Login with valid/invalid username	Pass — empty and non-letter usernames rejected client-side	Figure 2.1
Successful connection to server	Pass — status turns green, chat window opens	Figure 2.2

Test Case	Result	Evidence
Multiple simultaneous clients (h2–h5)	Pass — all 4 clients connected and exchanged messages concurrently	Figures 2.3, 2.6
Broadcast messaging	Pass — message from one client visible to all others, colored distinctly	Figure 2.4
Private messaging (/msg + double-click)	Pass — message delivered only to the intended recipient	Figure 2.5
Online user list auto-update	Pass — list refreshes on every join/leave//list	Figures 2.6, 2.7
User join notification	Pass — "has joined the chat" broadcast to existing clients	Figure 2.6
User leave / disconnect	Pass — "has left the chat" broadcast, socket closed cleanly	Figure 2.7
GUI responsiveness during receive	Pass — input box remained usable while messages streamed in	Observed during live testing

7. Wireshark Verification

Traffic between the clients (h2–h5) and the server (h1) was captured on the Mininet switch interface with the display filter `tcp.port == 5000`. The four required events — client connection, broadcast message, private message, and client disconnection — are shown below with the corresponding frame numbers from the capture.

7.1 Client Connection

Frames 9–11 show the TCP three-way handshake (SYN, SYN-ACK, ACK) as a client opens a connection to the server on port 5000.

9	19.202743161	10.0.0.2	10.0.0.1	TCP	74	38170 → 5000 [SYN] Seq=0 Win=42340 Len=0 MSS=1460 SACK_PERM TSval=3070901994 TSecr=0 WS=512
10	19.202799894	10.0.0.1	10.0.0.2	TCP	74	5000 → 38170 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=176065998 TSecr=3070901994 WS=512
11	19.205719960	10.0.0.2	10.0.0.1	TCP	66	38170 → 5000 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=3070902012 TSecr=176065998

Figure 7.1 — TCP three-way handshake establishing a new client connection (frames 9–11)

Frame 32 shows the client's very first application-level payload after the handshake — the username ("Rahul") being sent to the server, which the server uses to register the client in its online-users table.

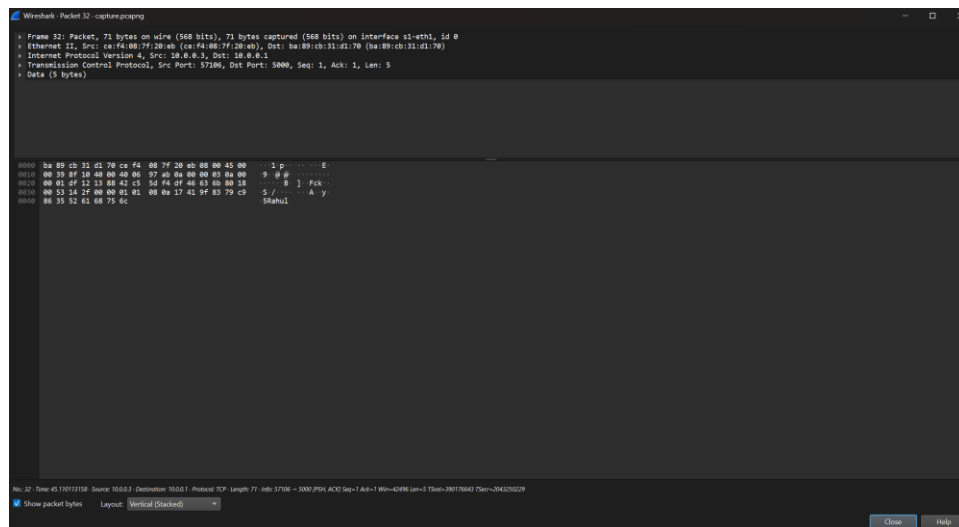


Figure 7.2 — Packet detail for frame 32: the client's username payload ("Rahul") sent immediately after connecting

7.2 Broadcast Message

Frames 34–35 show the server pushing a broadcast notification ("has joined the chat") out to the other already-connected clients — the same broadcast() path used for ordinary chat messages, each delivered as an individual PSH/ACK segment to every other client's socket.

34	45.116166843	10.0.0.1	10.0.0.2	TCP	101 5800 → 38170 [PSH, ACK] Seq=157 Ack=16 Win=43520 Len=35 TSval=176091912 TSecr=3070902136
35	45.116177288	10.0.0.1	10.0.0.3	TCP	101 5800 → 57106 [PSH, ACK] Seq=1 Ack=6 Win=43520 Len=35 TSval=2043250236 TSecr=390176643

Figure 7.3 — Server broadcasting a message to multiple connected clients (frames 34–35)

7.3 Private Message

A private message was sent with /msg Pipal Hello Pipal from Pedro (10.0.0.5). Unlike a broadcast, the server delivers this only to Pipal's socket (10.0.0.2) and separately acknowledges it back to the sender — it is never sent to Rahul or Pritam. The relevant frames, decoded from the capture, are:

Frame	Src → Dst	Payload
130	10.0.0.5 (Pedro) → 10.0.0.1 (server)	/msg Pipal Hello Pipal
131	10.0.0.1 (server) → 10.0.0.2 (Pipal) only	[PM from Pedro] Hello Pipal
132	10.0.0.1 (server) → 10.0.0.5 (Pedro) only	[PM to Pipal] Hello Pipal

The key point of interest is that frame 131 is addressed only to Pipal's IP (10.0.0.2) and no equivalent frame is sent to Rahul (10.0.0.3) or Pritam (10.0.0.4), which confirms the message stayed private at the network level rather than being broadcast.

7.4 Client Disconnection

Frame 160 is Pedro's client sending /quit. Frames 161–162 (visible via the FIN,ACK filter) show the normal TCP four-way close between the server and Pedro's socket. Frames 164–166 show the server then broadcasting "Pedro has left the chat" to the remaining connected clients.

197	243.993867160	10.0.0.2	10.0.0.1	TCP	71 38170 → 5000 [PSH, ACK] Seq=41 Ack=600 Win=42496 Len=5 TSval=3071126801 TSecr=176289318
198	243.993970763	10.0.0.1	10.0.0.2	TCP	66 5000 → 38170 [FIN, ACK] Seq=600 Ack=46 Win=43520 Len=0 TSval=176290789 TSecr=3071126801
199	243.994037214	10.0.0.2	10.0.0.1	TCP	66 38170 → 5000 [FIN, ACK] Seq=46 Ack=601 Win=42496 Len=0 TSval=3071126801 TSecr=176290789

Figure 7.4 — TCP FIN/ACK teardown as a client disconnects (frames 197–199 shown; the same pattern applies to Pedro's disconnect at frames 160–162)

8. Reflection Answers

Why should networking logic and GUI code be separated?

Keeping the socket/threading code (ChatConnection) independent of Tkinter makes the networking layer reusable and testable on its own — it could be swapped under a different UI (e.g. a web front-end) without any change, and it can be unit-tested by feeding it a fake socket without ever creating a Tk window. It also avoids accidental Tkinter calls from a background thread, which Tkinter does not support safely; the only bridge between the two layers is a thread-safe queue.Queue, which keeps the boundary explicit and enforced.

Why is a background thread required?

socket.recv() is a blocking call — it does not return until data arrives or the connection closes. If it were called on the same thread that runs Tkinter's mainloop(), the entire window (typing, clicking Send, redraw) would freeze until the next message arrived. Running recv() on a separate daemon thread lets the GUI thread stay free to process user input and redraw the window at all times, while the receiver thread only ever communicates back via the thread-safe queue.

Which components from Assignment 5 were reused?

The entire server (server.py) was reused without any modification — its threading model, client registry, broadcast/private-message logic, /list and /stats commands, chat-history CSV logging, and event logging were all kept exactly as they were. On the client side, the underlying TCP protocol (the same message formats and

commands) was preserved; only the terminal `input()/print()` loop was replaced with the `ChatConnection` class plus Tkinter widgets.

How did the GUI improve usability?

The GUI removes the need to remember exact command syntax for common actions — online users are visible in a live-updating list, double-clicking a name auto-fills a private message, and typing `/msg` triggers username autocomplete. Color-coded message types (system, broadcast, private, self, error) make the conversation far easier to scan than a single scrolling terminal stream, and a persistent status indicator plus a dedicated Disconnect button make the connection state obvious at all times.

Suggested future enhancement

Add TLS (wrapping the socket with `ssl.wrap_socket` / an `SSLContext`) so that usernames and message contents are encrypted in transit — as seen in the Wireshark capture, all chat text is currently sent as plain-text TCP payload and is trivially readable by anyone capturing traffic on the network.

9. Conclusion

This assignment successfully converted the Assignment 5 terminal chat client into a full graphical desktop application while reusing the Assignment 5 server without modification. The GUI client (`client_gui.py`) supports login validation, broadcast and private messaging, a live online-users panel, join/leave notifications, and non-blocking background message reception via a dedicated receiver thread and a thread-safe queue. Testing with four simultaneous clients inside a Mininet single-switch topology confirmed correct behavior for all required features, and packet capture with Wireshark verified the underlying TCP handshakes, broadcast delivery, private-message delivery, and connection teardown at the protocol level. The result is a more usable and observable chat application built directly on top of the previously validated networking core.